Course: DevOps with CI/CD for Edge
Lecture: **Module 02 – Automated Testing**

Your tasks for the exercise:

Preparation

- Fork and clone the Module 2 repository: https://github.com/nit-bogdan-pavkovic/m02.devops.training
- Python should be installed
- Each exercise is stored in a different folder; open the folders in order. You can find instructions in the README.md file.

Exercise 1: TDD methodology

1. Open `test_factorial.py` and examine its contents
2. Add a few more tests in the same file. We're testing a function called `factorial`, which calculates the factorial value of any number. For example `factorial(4) = 4*3*2*1 = 24`. Think of edge cases: what is the factorial of 0? What is the factorial of a negative number?
3. **(RED STAGE)** Run `python test_factorial.py` you should get an error message like this:
```
F
======================================================================
FAIL: test_factorial_of_0 (__main__.TestFactorial.test_factorial_of_0)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/Users/tom/r/devops-course/module3/tdd/test_factorial.py", line
7, in test_factorial_of_0
    self.assertEqual(factorial(0), 1)
    ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^
AssertionError: None != 1
----------------------------------------------------------------------
Ran 1 test in 0.001s

FAILED (failures=1)
```
4. **(GREEN STAGE)** Open `factorial.py` and implement the solution until ALL the tests have passed one by one.
5. **(REFACTOR STAGE)** Improve the solution in `factorial.py` to make it more general or cover edge cases.

Exercise 2: Unit testing

1. Open and examine `math_sum.py`
2. Open `test_math_sum.py`
3. Add a few test cases (at least 4 more). Try to think about edge cases: what would happen when adding negative numbers, big numbers, floating-point numbers, etc.?
4. Run `python test_math_sum.py` to confirm the unit tests pass

Exercise 3: Integration testing

1. Open `datastore.py` and `service.py` files and examine how they work
2. `test_integration.py` and examine its contents
3. Remove the `pass` keyword and implement an integration test using **self.assertEqual** method.
4. Confirm that the test works using the command *`python test_integration.py`*

Exercise 4: End-to-end testing

1. Create a virtualenv with: `python -m venv venv`
2. Activate the virtualenv: `source venv/bin/activate`
3. Install the dependencies using `pip install -r requirements.txt`
4. Open `app.py`, and examine this *Flask* application
5. Open `test_e2e.py`, examine the test structure
6. Implement two tests:
    a) Perform a GET request to `http://127.0.0.1:5000/`, then check that the status code is 200 and the response body contains the string "Welcome"
    b) Perform a POST request on `http://127.0.0.1:5000/add`. Send a JSON message in the request and check that the status code is 200 and the response data in the body is correct.
    c) Hints: use the curl command line tool to perform both tests. Alternatively, you can write a Python test that uses the requests package to perform the HTTP requests.

Exercise 5: Test reports

1. Create a virtualenv with: `python -m venv venv`
2. Activate the virtualenv: `source venv/bin/activate`
3. Install the dependencies using `pip install -r requirements.txt`
4. `Run pytest and see that some tests are failing. This is ok.`
5. `Run pytest -junitxml=reports.xml to create an XML file with the reports`

Exercise 6: CI pipeline

1. Add this project to your Semaphore account
2. In the pipeline, use the container image: **`registry.semaphoreci.com/python:3.12.1`**
3. Create a pipeline with 4 blocks:
    d) Install dependencies
    e) Unit tests
    f) Integration tests
    g) End-to-end tests
    h) Test reports
4. Ensure the pipeline runs without errors. If a job fails, open the job log to see the error. Discuss possible solutions.

Setting up test reports

During exercise 6, where we configure the pipeline, you will need the following commands in job number 5 (Test reports)

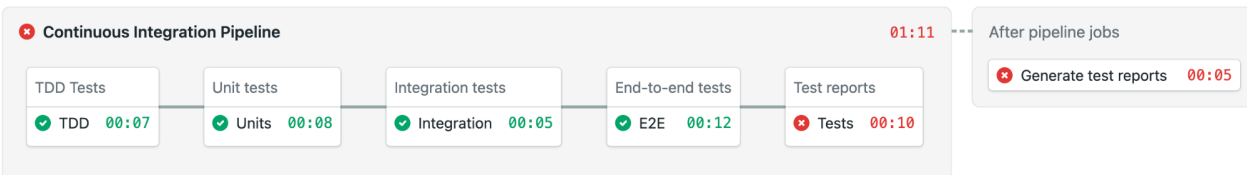
```
# enter the directory
cd 5-test-reports
# install dependencies
pip install -r requirements.txt
pytest --junitxml=reports.xml
[[ -f report.xml ]] && test-results publish report.xml
```

Next, create an "After pipeline job" with the following command:

`test-results gen-pipeline-report`

Tips

- Check **pipeline.png** in this folder to see an example.
- If you have trouble running `app.py` in CI, try using ***`nohup python app.py &`***



Bonus

- If you have time, you may implement optimization techniques discussed in Module 01 (caching dependencies).